

Wrench Transport

Retained contact impulses and coupled body response
for rigid-body simulation

Joshuah Rainstar

3 October 2026

Abstract

Wrench Transport is a rigid-body engine that represents contact as a persistent field of transmitted impulses. Each time step solves for the coupled linear and angular velocities of the active bodies. Friction-cone projection gives a strongly convex inner velocity problem; sparse block elimination communicates its response through the contact graph; retained impulses and load-scaled regularization connect successive solves. The engine supports convex and compound geometry, static bodies, restitution, rolling resistance, and sleeping contact islands. We evaluate the C++20 implementation on mixed-shape packing and mechanical acceptance tests, and compare it with MuJoCo and Rapier using common geometry, settling criteria, and geometric overlap measurements. Across three common packing scenes, the 60 Hz configuration settles every scene with a largest final overlap of 0.0136 mm and a median compute cost of 0.336 s for eight simulated seconds. Default Rapier settles the same scenes in 0.227 s with 6.975 mm largest final overlap. A separate seven-scene study measures a mean native step time of 0.7184 ms on an Apple M4.

1 Introduction

A new load on a rigid structure changes the forces required throughout its supporting contacts. A simulation must resolve that coupled response while maintaining unilateral contact, friction, and rotational balance. The central design choice in Wrench Transport is to preserve the impulse already transmitted at each contact and solve for the response of the connected bodies within the time step.

The motivating intuition treats contact as transport. An influence enters the contact network, its consequences pass through the connected bodies, and the resulting field must support a consistent motion. The useful unit of that exchange is a *wrench*: the force and torque applied to one rigid body,

$$\mathcal{W} = \begin{bmatrix} F \\ \tau \end{bmatrix}, \quad F = \sum_c f_c, \quad \tau = \sum_c (p_c - x) \times f_c. \quad (1)$$

Its dual variable is the body’s twist, consisting of linear and angular velocity. Eliminating a body’s unknown twist condenses both its load and its response onto the remaining contact boundary. The supporting bodies therefore respond to the structure they carry, including its rotational coupling.

Two properties guide the engine. First, a static load must persist: numerical transport cannot attenuate the impulse that holds a body at rest. Second, a contact’s numerical scale should reflect the mass it supports, which may greatly exceed the mass of either adjacent body. Retained impulses address the first property, while carried-mass and previous-load scaling address the second.

The engine combines these choices with a velocity-space convex inner solve. Related primal contact formulations include the Semi-Analytic Primal solver (SAP) of Castro, Permenter, and Han [1]. Wrench Transport uses a retained-impulse outer update and a load-dependent regularization, with a sparse six-dimensional body-block factorization for each Newton system.

2 One contact field, one coupled response

Let $V \in \mathbb{R}^{6n}$ collect the awake bodies' twists, V^* denote their free-motion twists after external forces, and $M \succ 0$ their block mass/inertia matrix. For contact c , $J_c V$ is relative contact velocity; λ_c is its retained impulse; $R_c > 0$ is a scalar algorithmic regularization; and $\hat{v}_c$ is a target velocity held fixed during an inner solve. The friction cone is

$$K_c = \{(\gamma_t, \gamma_n) : \gamma_n \geq 0, \|\gamma_t\| \leq \mu_c \gamma_n\}. \quad (2)$$

Writing P_{K_c} for Euclidean projection onto that cone, the contact portion of an inner problem is

$$y_c(V) = \lambda_c - \frac{J_c V - \hat{v}_c}{R_c}, \quad \gamma_c(V) = P_{K_c}(y_c(V)), \quad (3)$$

$$\Phi(V) = \frac{1}{2}(V - V^*)^T M (V - V^*) + \sum_c \frac{R_c}{2} \|P_{K_c}(y_c(V))\|^2. \quad (4)$$

Its gradient expresses momentum balance directly:

$$\nabla \Phi = M(V - V^*) - \sum_c J_c^T \gamma_c. \quad (5)$$

On a smooth projection branch, the Newton matrix is

$$H = M + \sum_c J_c^T G_c J_c, \quad G_c = R_c^{-1} D P_{K_c}(y_c) \succeq 0. \quad (6)$$

Thus each fixed inner problem is strongly convex. The implementation uses branch derivatives at contact transitions, a line search, and sparse 6×6 block elimination. Additional bounded rows implement rolling resistance and the hold; Equation (4) describes the contact core.

2.1 What the macro-frame becomes

Partition a Newton system into a body to eliminate and its remaining boundary:

$$\begin{bmatrix} A & B \\ B^T & D \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} a \\ b \end{bmatrix} \implies (D - B^T A^{-1} B)v = b - B^T A^{-1} a. \quad (7)$$

The Schur complement transports both load and response. Back-substitution then returns the boundary's response to the eliminated body. This is the precise realization of “everything above contributes now.” It is standard elimination, applied to the contact graph; the implementation stores block transfers and uses a minimum-degree ordering.

A body contributes six degrees of freedom; a condensed cluster retains the coupled degrees of freedom on its boundary. The cost depends on graph fill and elimination order. This use of structured body response connects to classical constrained-dynamics algorithms [2].

2.2 Retention and carried load

After an inner solve, retained rows set $\lambda_c \leftarrow \gamma_c$. The next pass also updates the sliding normal target with a $-\mu_c \|v_{t,c}\|$ shift. Regularization is scaled by the largest of local effective mass, an estimated carried-mass share, and mass inferred from last frame's normal impulse $\lambda_n / (\|g\| h)$. This makes the numerical scale responsive to the load a contact actually supports.

At a consistent resting state, $V = 0$, targets and retained impulses agree, and the gradient vanishes: support can be a fixed point without deliberately creeping through a soft spring. The implementation terminates the outer iteration on retained-impulse and sliding-shift changes, with a maximum of six passes. Inner residual tolerances tighten as the retained field converges.

3 Engine architecture

The standalone implementation is a C++20 static library. The public interface provides shape cooking, dynamic and static body creation, pose and velocity queries, fixed-step simulation, contact inspection, raycasting, a bounded crane hold, and rest-state queries. Simulation code has no renderer or window-system dependency.

Geometry and collision. Input point clouds are cooked into convex hulls with polygonal faces, edge adjacency, mass, center of mass, and inertia. A body may contain multiple convex hulls to represent compound geometry. The narrow phase uses face and edge separating axes, Gauss-map edge pruning, and clipped contact manifolds checked against supporting witnesses. A speculative margin includes the bodies’ estimated motion. Cached separated pairs can be rejected while a conservative movement bound remains smaller than their separation; sufficiently stationary touching pairs reuse their local manifold.

Contact and integration. Each active body’s free twist includes gravity and configured damping. Contact rows contribute friction-cone impulses; separate bounded rows model rolling resistance and the crane hold. Restitution is activated at contact closure above a speed threshold. After the coupled solve, the engine integrates position and orientation and retains the resulting contact impulses for the next frame.

Rest and wake. Contacts connect bodies into islands. An island sleeps when every free body remains below the linear and angular thresholds for the required quiet interval. Contact from an awake body, removal of supporting geometry, or an explicit pose change wakes the relevant bodies. The performance comparisons disable sleeping to expose the cost and stability of continued contact simulation.

Computational structure. A body contributes a 6×6 inertia block, and a contact couples at most two body blocks. Symbolic ordering is reused when the contact graph is unchanged. Within a frame, a numerical factor can be reused while the relevant contact branches remain unchanged; sliding branches generally require refactorization. Small vector and quaternion operations are inline, preserving the arithmetic while allowing optimization across call sites. Floating-point contraction is disabled in the measured build.

4 Experimental design

The comparison scene contains 64 dynamic convex bodies, drawn from eight shape classes: boxes, sphere approximations, cylinders, cones, thin sheets, prisms, general polyhedra, and irregular rocks. The bodies begin on a four-layer grid with seeded orientations inside a 0.37×0.37 m container. Every engine receives the same hull vertices, initial poses, density of 2600 kg/m^3 , friction coefficient of 0.5, and gravity of 9.81 m/s^2 . Restitution, damping, and rolling resistance are disabled for this workload.

Each run advances eight simulated seconds and samples motion at 60 Hz. A body-speed measure combines translation and rotation as $\|v\| + r \|\omega\|$, where r is its hull radius. Settling is the first sample after which the maximum body speed stays below 4 mm/s through the end of the run. Final interpenetration is evaluated with one independent exhaustive separating-axis calculation on the exported hull poses, including body pairs, walls, and floor.

Wrench Transport runs at 60 and 120 Hz. Rapier 0.21.0 is evaluated at 60 Hz with four solver iterations, both its default length unit and a 0.05 m length unit, and at 240 Hz with eight iterations and the smaller length unit. MuJoCo 3.2.3 uses Newton solving, elliptic friction cones, and multi-contact convex collision at 60 and 120 Hz, plus a 480 Hz configuration with a 0.005 s contact time constant. These configurations expose the cost–accuracy tradeoff rather than equating iteration counts across different solvers.

5 Evaluation

The native performance study uses seven scene seeds and seven measured repetitions after a warm-up. With sleeping disabled, Wrench Transport averages 0.7184 ms per 60 Hz step across these 64-body scenes. Moving frames average 2.7550 ms; quiet but active frames average 0.2631 ms. The measurement surrounds the simulation step and excludes shape cooking, result serialization, and rendering.

The cross-engine comparison uses seeds 1, 7, and 19, one warm-up and three measured repetitions per configuration, with rotating execution order. The hardware is an Apple M4 Mac mini running macOS 26.5. Wrench Transport uses an AppleClang 21 Release build; MuJoCo runs its native solver through Python; Rapier runs its WebAssembly distribution through Node. Timings include the respective step-call boundary. Per-run records retain runtime, settling time, final overlap, and remaining motion.

Configuration	Compute (s)	Overlap (mm)	Settled	Late speed (m/s)
Wrench Transport, 60 Hz	0.336	0.0136	3/3	4.18×10^{-6}
Wrench Transport, 120 Hz	0.481	0.0244	3/3	9.98×10^{-6}
Rapier, 60 Hz	0.227	6.975	3/3	1.28×10^{-5}
Rapier, 60 Hz, scaled	0.181	9.794	0/3	0.236
Rapier, 240 Hz, scaled	0.653	6.330	2/3	0.285
MuJoCo, 60 Hz	6.291	30.637	0/3	2.618
MuJoCo, 120 Hz	10.075	6.064	0/3	0.843
MuJoCo, 480 Hz, stiff	16.400	0.241	0/3	0.0446

Compute cost is the median of nine measured runs, each advancing eight simulated seconds. Overlap is the largest final penetration across the three seeds and their repetitions. Settled counts scenes satisfying the sustained 4 mm/s criterion; late speed is the largest body-speed measure during the last second across all runs. Three bodies leave the container in the worst MuJoCo 60 Hz case; all other configurations retain every body.

Wrench Transport at 60 Hz settles all three scenes in 1.20–1.37 simulated seconds. Its median compute time corresponds to approximately 23.8 simulated seconds per compute second. Default Rapier is faster at approximately 35.3 times real time and also reaches rest, with a larger retained geometric overlap. Reducing Rapier’s length unit alone lowers runtime but leaves continuing motion in these scenes. Increasing its rate and iterations recovers settling for two of the three scenes.

MuJoCo’s stiff 480 Hz configuration substantially reduces overlap relative to its lower-rate configurations, reaching a maximum of 0.241 mm. It retains residual motion above the settling criterion over the eight-second observation interval. MuJoCo’s compliant contact model and Rapier’s tolerance-scaled iterative solver expose different accuracy–cost controls [3, 4]; the measured configurations are specified above so their operating points can be reproduced.

The 120 Hz Wrench Transport run does not uniformly improve final geometric overlap: it follows a different contact trajectory and packing arrangement. Its largest overlap remains below 0.025 mm. Both rates reach rest in every scene, indicating that the tested 60 Hz operating point already provides stable support for this workload.

5.1 Mechanical and numerical checks

The native acceptance suite includes resting rocks, tall stacks, large mass ratios, overhangs, friction inclines, drops, crane holds and releases, support removal, wake propagation, compound bodies, restitution, and deterministic replay. Rest tests span round, flat, and jagged hulls over 50 seeds and 600 simulated seconds per shape class. Tall-stack tests use 15 bodies over 20 seeds; compound-stack tests use ten seeds.

An independent dense Gaussian-elimination comparison checks 80 coupled positive-definite systems against the block solver. The maximum equation residual is 2.398×10^{-14} and maximum solution difference is 3.575×10^{-14} . A separate exhaustive SAT oracle checks 6,000 near-contact convex pairs, with zero missed overlaps, false contacts beyond the margin, or invalid normals in that sample.

6 Discussion

Wrench Transport’s design emphasizes persistent support and coupled response. Retaining impulses makes the supported state available to the next frame, while carried-load scaling keeps the contact regularization sensitive to the structure above a contact. Sparse elimination resolves the linearized response across the connected graph. In the mixed-convex packing comparison, the 60 Hz engine combines sustained rest and sub-0.014 mm final overlap at a median cost of 0.336 s per eight simulated seconds. The default Rapier configuration provides the lowest cost among configurations that settle all three scenes, while Wrench Transport provides the smallest maximum final overlap. This places the engine at a useful accuracy–cost operating point for small rigid contact assemblies.

The three comparison seeds sample a small family of dense packing problems. They do not cover articulated robotics, large open worlds, or sustained high-speed impacts. The implementation currently uses discrete collision with speculative contacts. Continuous collision detection, general joint systems, and deformable bodies are outside its present feature set. Broad-phase enumeration and symbolic adjacency use quadratic structures; dense contact graphs can also increase factorization fill. Those properties define useful directions for larger-scale and high-impact workloads. The inner problem is strongly convex for fixed retained impulses and targets; the outer friction-shift update has a finite pass budget, so its convergence is assessed empirically here.

7 Availability

The C++20 source, CMake package, mechanical and numerical tests, scene generator, comparison runners, and per-run records are available in the BFFT repository. The library exports the `Wrench::Physics` build target and the `zc::phys` C++ interface. A minimal example constructs a body, advances the world, and checks its settled state.

https://github.com/falseywinchnet/bfft/tree/codex/wrench-engine-study/experiments/wrench_transport

References

- [1] A. Castro, F. Permenter, and X. Han. *An Unconstrained Convex Formulation of Compliant Contact*. IEEE Transactions on Robotics, 2022. DOI: 10.1109/TRO.2022.3209077. <https://arxiv.org/abs/2110.10107>.
- [2] D. Baraff. *Linear-Time Dynamics Using Lagrange Multipliers*. SIGGRAPH, 1996, pp. 137–146. <https://www.cs.cmu.edu/~baraff/papers/sig96.pdf>.
- [3] MuJoCo 3.2.3 documentation. *Computation: soft contact model and solvers*. <https://mujoco.readthedocs.io/en/3.2.3/computation/>.
- [4] Dimforge. *Rapier integration parameters*. https://rapier.rs/docs/user_guides/javascript/integration_parameters/.